

GenAI: Test Case Development

This module focuses on how **GitHub Copilot and Generative AI** accelerate and improve **test case creation, data generation, prioritization, and documentation**.

Role of GitHub Copilot in Test Case Development

GitHub Copilot helps testers:

- Generate unit, API, and UI test cases
 - Create edge and negative test cases
 - Generate synthetic and augmented test data
 - Create test scenarios
 - Prioritize test cases
 - Auto-document test cases
-

Generating Test Cases: Example

Application: User Registration

```
def register(username, password):  
    if len(password) < 6:  
        return "Weak password"  
    return "Registered"
```

Prompt

```
# Generate pytest test cases for register function
```

Copilot Output

```
def test_register_valid():  
    assert register("john", "abcdef") == "Registered"  
  
def test_register_short_password():  
    assert register("john", "123") == "Weak password"
```

Generating Synthetic Test Data: Example

Synthetic data = artificially created data.

```
# Generate synthetic user data
users = [
    {"username": "user1", "email": "user1@test.com"},
    {"username": "user2", "email": "user2@test.com"}
]
```

Generating Augmented Test Data: Example

Augmented data = variations of existing data.

```
original = {"username": "john"}
augmented = [
    {"username": "john1"},
    {"username": "JOHN"},
    {"username": "john_test"}
]
```

Demo: Generating Synthetic and Augmented Datasets Using GitHub Copilot

Step 1

Create `test_data.py`

Step 2

Type:

```
# Generate synthetic and augmented datasets for login testing
```

Step 3

Copilot generates multiple datasets.

Creating Test Scenarios: Example

Scenario:

User registers → logs in → updates profile.

Given user is registered

When user logs in

Then dashboard is displayed

Test Case Prioritization: Example

GenAI prioritizes based on risk.

Test Case	Priority
Login valid	High
Password reset	High
UI color	Low

Automating Test Case Documentation: Example

```
# Add docstring for test cases
```

Copilot generates documentation.

Demo: Automating Test Case Prioritization and Documentation Using GitHub Copilot

Step 1

Create `tests.py`

Step 2

Type:

```
# Prioritize and document test cases
```

Step 3

Copilot adds priorities and docstrings.

Benefits

- Faster development
 - Better coverage
 - Reduced manual effort
-

Limitations

- Needs review
 - Domain-specific logic may be missed
-

Quick Check

Q1. What is synthetic test data?

Artificially generated data

Q2. True/False

Copilot can generate test scenarios.
True

Q3. Best practice?

Review AI-generated tests

Summary

Generative AI simplifies test case development by:

Generating tests

Creating data

Prioritizing & documenting

But **human validation remains essential.**
